

DEPARTMENT OF DECISION SCIENCE

FACULTY OF BUSINESS

UNIVERSITY OF MORATUWA

SEMESTER 4

DA2111 – Statistical & Machine Learning

Individual Assignment

Lecturers: Mr. Maninda Edirisooriya

Mr. Maninda Edirisooriya

Due date of submission: November 10, 2024

Name:Bavindu Gunasinghe

Index:226044G

Table of Contents

1. Data Set Selection and Associated Task 3

2. Problem Definition 3

Target Variable 3

Research Question 4

3. Data Exploration and Preprocessing 4

Data Exploration 4

Data Preprocessing 6

4. Exploratory Data Analysis 9

5. Model Selection 13

Machine Learning Algorithms for Regression 13

6. Model Training and Evaluation 15

7. Hyperparameter Tuning 21

8. Comparison and Conclusion 25

Challenges Encountered and Solutions 26

1. Data Set Selection and Associated Task

Based on final letter of index number customer churn data set was selected. The dataset selected contains information on customer behavior and usage patterns within a telecom company. Each record in the dataset represents a customer and includes various attributes related to their service usage, demographic details, and interactions with the company.

Some of the features include:

- Usage Metrics: Information about the frequency and duration of calls, SMS usage, and total seconds of use, providing insight into how actively each customer uses the telecom services.
- Subscription Information: Details like the length of time a customer has been subscribed to the service, the number of unique numbers they have called, and the charges they incur, which help to understand their relationship with the service.
- Demographic Information: Basic details such as the customer's age, which might play a role in usage patterns.

The assigned task is a regression problem. Specifically, we aim to predict the Customer Value, a continuous variable that represents the monetary or behavioral worth of a customer to the company.

2. Problem Definition

Target Variable

Customer Value is likely a measure of the potential revenue or overall worth that each customer brings to the company.

Features

To predict the **Customer Value**, use the following features as inputs based on the dataset provided. These features capture different aspects of customer behavior, service usage, and demographic information that might influence the overall value of a customer to the business.

- **Age**: The age of the customer.
- **Subscription Length**: The duration for which the customer has been subscribed to the service.

- **Seconds of Use**: The total amount of time the customer has spent using the service in seconds.
- **Frequency of SMS**: The number of SMS messages sent by the customer.
- **Frequency of Use**: How frequently the customer uses the service (e.g., daily, weekly).
- **Distinct Called Numbers**: The number of unique phone numbers the customer has called, indicating the diversity of their interactions.
- **Call Failure**: The number of failed call attempts, which could reflect network issues or user dissatisfaction.
- **Frequency of Call Transfer**: The number of times calls are transferred, possibly indicating complexity in interactions or customer needs.
- **Charge Amount**: The amount the customer has been charged for services, which may correlate with higher or lower customer value.
- **Service Plans**: Information about specific plans or features of the service the customer is subscribed to (if included in the dataset).

Research Question

"How can predict the Customer Value based on customer demographics, service usage patterns, and interaction with the service?

3. Data Exploration and Preprocessing

Data Exploration

In data exploration, focus on understanding the structure and characteristics of the dataset. This process will help identify any issues, patterns, and relationships within the data that can impact the modeling process.

Start by loading the dataset and displaying its structure to understand the types of variables and their initial values.

```
import pandas as pd

# Load the dataset
data = pd.read_csv('/content/drive/MyDrive/Customer Churn.csv')
```

```python
# Display the first few rows of the dataset
print(data.head())

# Display information about the dataset structure
print(data.info())
```

```
Call  Failure  Complains  Subscription  Length  Charge  Amount  \
0              8          0                    38               0
1              0          0                    39               0
2             10          0                    37               0
3             10          0                    38               0
4              3          0                    38               0

   Seconds of Use  Frequency of use  Frequency of SMS  \
0            4370                71                 5
1             318                 5                 7
2            2453                60               359
3            4198                66                 1
4            2393                58                 2

   Distinct Called Numbers  Age Group  Tariff Plan  Status  Age  \
0                       17          3            1       1   30
1                        4          2            1       2   25
2                       24          3            1       1   30
3                       35          1            1       1   15
4                       33          1            1       1   15

   Customer Value  Churn
0         197.640      0
1          46.035      0
2        1536.520      0
3         240.020      0
4         145.805      0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3150 entries, 0 to 3149
Data columns (total 14 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Call  Failure            3150 non-null   int64
 1   Complains                3150 non-null   int64
 2   Subscription  Length     3150 non-null   int64
 3   Charge  Amount           3150 non-null   int64
 4   Seconds of Use           3150 non-null   int64
 5   Frequency of use         3150 non-null   int64
 6   Frequency of SMS         3150 non-null   int64
 7   Distinct Called Numbers  3150 non-null   int64
 8   Age Group                3150 non-null   int64
 9   Tariff Plan              3150 non-null   int64
 10  Status                   3150 non-null   int64
 11  Age                      3150 non-null   int64
 12  Customer Value           3150 non-null   float64
 13  Churn                    3150 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 344.7 KB NoneNone
```

Potential Challenges

- It's essential to identify any missing values in the dataset, as they can affect model performance.

```
# Check for missing values
missing_values = data.isnull().sum()
print("Missing values in each column:\n",missing_values[missing_values > 0])
```

```
Missing values in each column:
 Series([], dtype: int64)
```

Since there are no missing values, further preprocessing steps can proceed.

Data Preprocessing

- **Handling Missing Values**

  If identified any missing values in the dataset, decide on appropriate methods to handle them (e.g., filling with mean/median for numerical features or mode for categorical features).

  However in this data set there were not any missing values, as shown in previous output.

- **Detecting and Handling Outliers**

  Outliers can distort the distribution of data and affect model performance, so we need to identify and handle them. Typically, we use box plots to visually detect outliers and then cap them to reduce their impact.

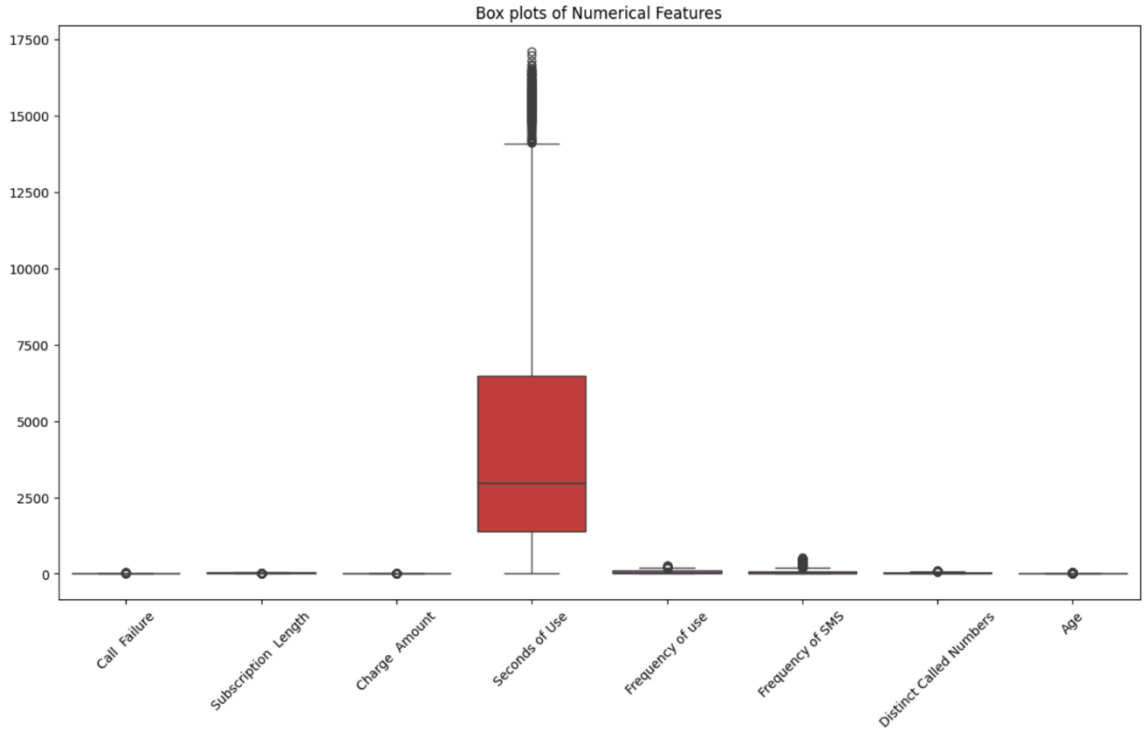
```
import matplotlib.pyplot as plt
import seaborn as sns

# List of numerical features
numerical_features = ['Call  Failure', 'Subscription  Length', 'Charge  Amount',
                      'Seconds of Use', 'Frequency of use', 'Frequency of SMS',
                      'Distinct Called Numbers', 'Age']

# Visualize all numerical features in a single box plot chart
plt.figure(figsize=(15, 8))
sns.boxplot(data=data[numerical_features])
plt.title('Box plots of Numerical Features')
```

```
plt.xticks(rotation=45)  # Rotate feature names for better readability

plt.show()
```



This plot indicates that **handling outliers** in "Seconds of Use" will likely be important for preprocessing, as these extreme values could skew model predictions. Additionally, the large range of values for this feature compared to the others may require **scaling** to ensure that all features contribute equally to model training.

Once we identify the outliers, we can cap values at a specific percentile range (e.g., 5th and 95th percentiles) to limit extreme values.

```
# Cap outliers at the 5th and 95th percentiles
for col in numerical_features:
    lower_limit = data[col].quantile(0.05)
    upper_limit = data[col].quantile(0.95)
    data[col] = np.where(data[col] < lower_limit, lower_limit,
data[col])
    data[col] = np.where(data[col] > upper_limit, upper_limit,
data[col])
```

- **Encoding Categorical Variables**

For categorical features, we need to convert them into numerical format. One common approach is One-Hot Encoding, which creates binary columns for each category.
Identify the categorical columns, then use pd.get_dummies() to encode them.
drop_first=True avoids multicollinearity by dropping the first category of each feature.

```
# Identify categorical columns
categorical_features = ['Complains', 'Age Group', 'Tariff Plan', 'Status']

# Perform one-hot encoding
data = pd.get_dummies(data, columns=categorical_features, drop_first=True)
```

- **Scaling Numerical Features**

Scaling is essential for many machine learning algorithms that are sensitive to the range of feature values. **Standardization** is commonly used to scale numerical features so they have a mean of 0 and a standard deviation of 1.

```
from sklearn.preprocessing import StandardScaler

# Re-identify numerical features after encoding
numerical_features = ['Call  Failure', 'Subscription  Length',
'Charge  Amount',
                      'Seconds of Use', 'Frequency of use', 'Frequency of
SMS',
                      'Distinct Called Numbers', 'Age']

# Initialize the scaler
scaler = StandardScaler()

# Apply standard scaling to numerical features
data[numerical_features] = scaler.fit_transform(data[numerical_features])
```

4. Exploratory Data Analysis

EDA will help us uncover relationships between these features and the target variable, as well as understand the data distribution and correlations.

- **Conduct EDA to Gain Insights into the Relationships within the Data**

```
# Basic statistics summary
print(data.describe())

# Check unique values in categorical columns
for col in data.select_dtypes(include='object').columns:
    print(f"{col}: {data[col].unique()}")
```

```
       Call  Failure  Subscription  Length  Charge  Amount  Seconds of Use
\
count    3150.000000           3150.000000     3150.000000     3150.000000
mean        7.377460             32.646984        0.853333     4439.774286
std         6.610619              7.942702        1.186116     4099.948579
min         0.000000             13.000000        0.000000       58.336250
25%         1.000000             30.000000        0.000000     1391.250000
50%         6.000000             35.000000        0.000000     2990.000000
75%        12.000000             38.000000        1.000000     6478.250000
max        22.000000             42.000000        4.000000    15016.663750

       Frequency of use  Frequency of SMS  Distinct Called Numbers  \
count       3150.000000       3150.000000              3150.000000
mean          67.562948         69.955647                22.585397
std           52.389794        102.070077                14.677658
min            1.000000          0.000000                 1.000000
25%           27.000000          6.000000                10.000000
50%           54.000000         21.000000                21.000000
75%           95.000000         87.000000                34.000000
max          184.166375        356.166375                51.000000

               Age  Customer Value        Churn
count  3150.000000     3150.000000  3150.000000
mean     31.388889      470.972916     0.157143
std       8.321284      517.015433     0.363993
min      25.000000        0.000000     0.000000
25%      25.000000      113.801250     0.000000
50%      30.000000      228.480000     0.000000
75%      30.000000      788.388750     0.000000
max      55.000000     2165.280000     1.000000
```

This code provides a summary of numerical features, helping identify the range, mean, and distribution of each.

- **Visualize Key Features and Their Distributions**

  Visualizing the distributions of key features helps us understand their spread, skewness, and presence of outliers. This is particularly helpful for identifying if any features are heavily skewed or if they have distinct groups or patterns.

  For a dataset like this, can focus on the distribution of important features like "Seconds of Use," "Charge Amount," "Subscription Length," and others that might correlate with "Customer Value."

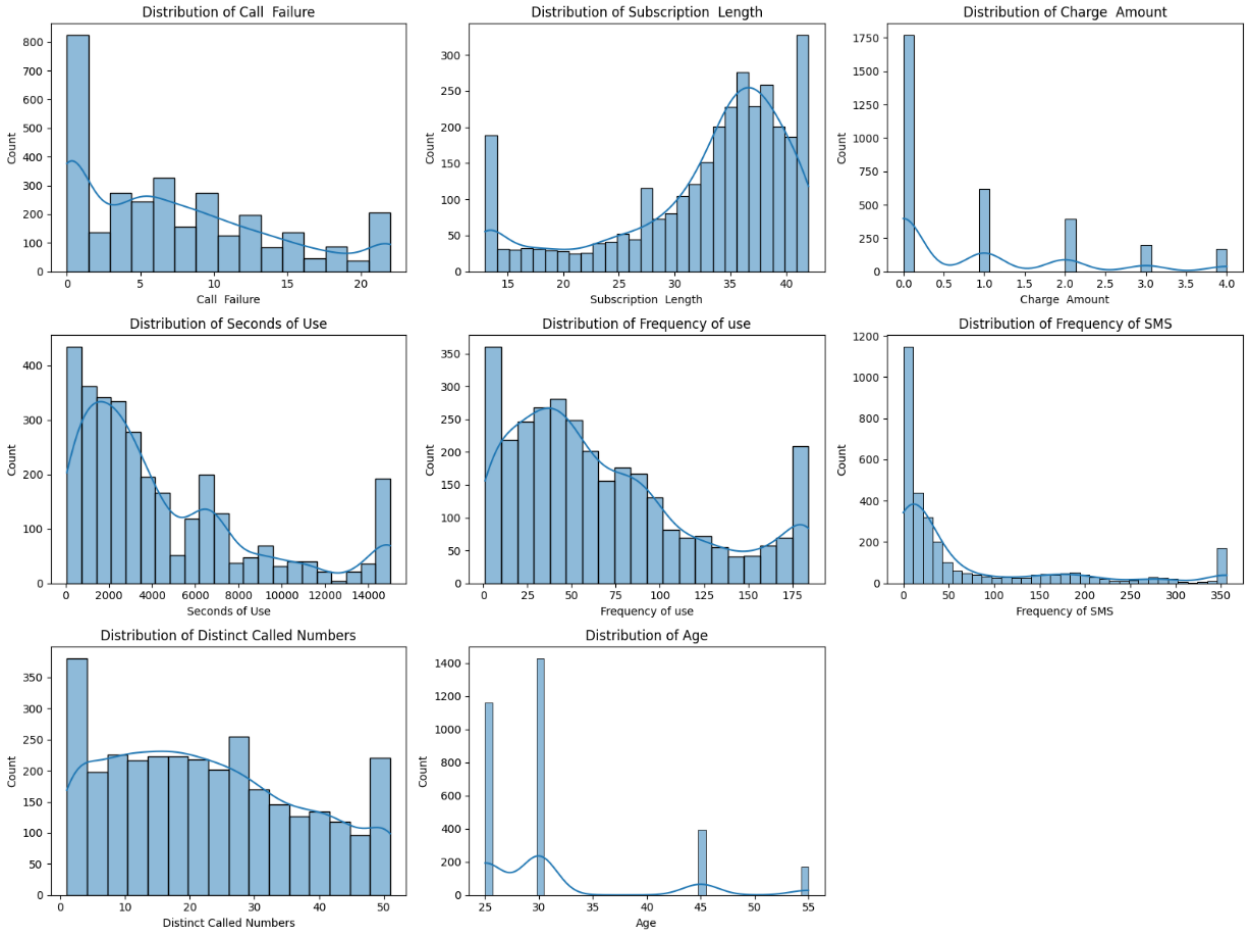
  This code generates histograms for each numerical feature to show their distributions.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Histograms for numerical features
# Correct column names with two spaces:
numerical_features = ['Call  Failure', 'Subscription  Length',
'Charge  Amount',
                      'Seconds of Use', 'Frequency of use', 'Frequency
of SMS',
                      'Distinct Called Numbers', 'Age']

plt.figure(figsize=(16, 12))
for i, feature in enumerate(numerical_features, 1):
    plt.subplot(3, 3, i)
    # Access the column using the corrected name:
    sns.histplot(data[feature], kde=True)
    plt.title(f'Distribution of {feature}')

plt.tight_layout()
plt.show()
```



- **Identify Potential Correlations between Variables**

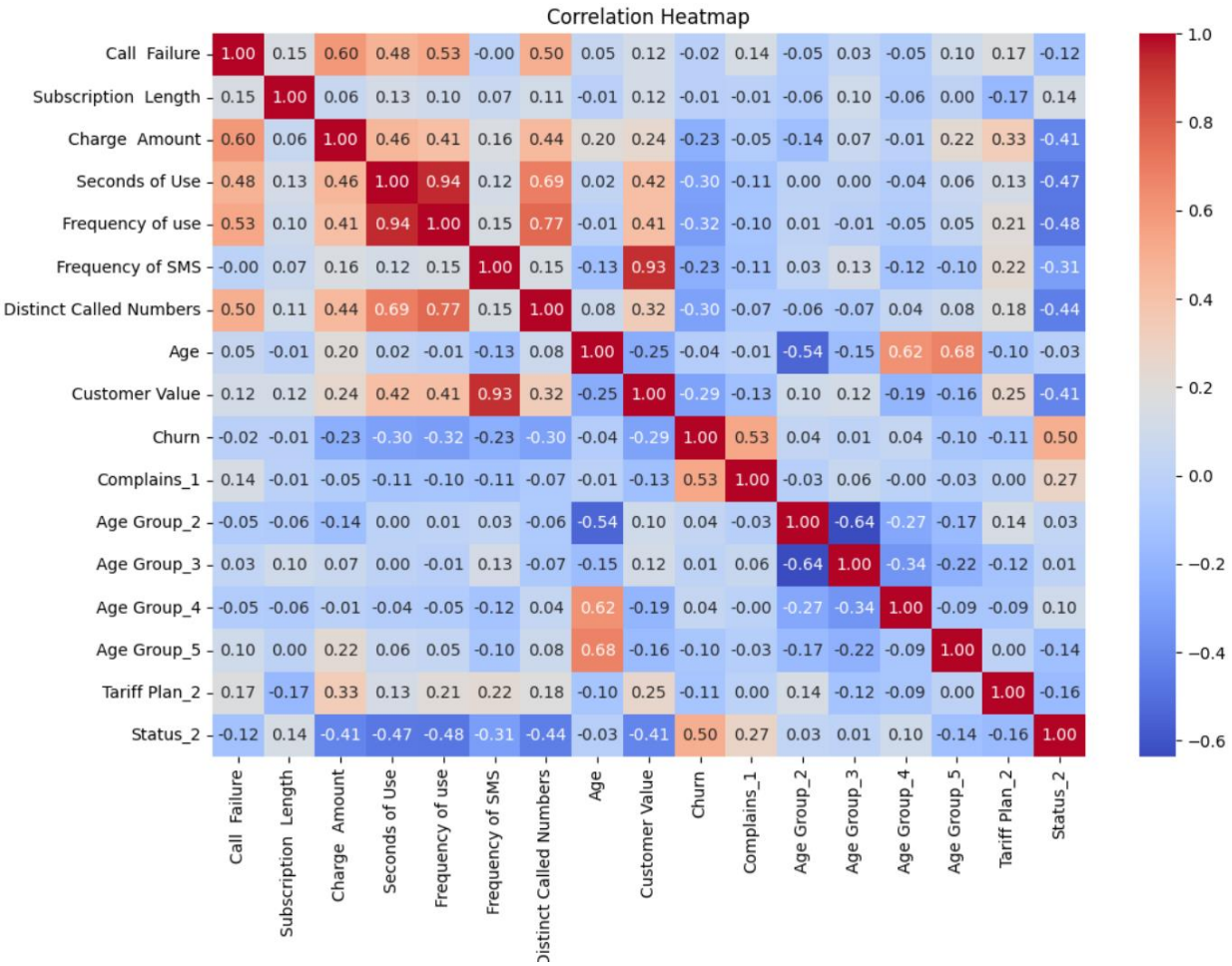
  Understanding correlations helps identify which features may be linearly related to others, potentially affecting model performance due to multicollinearity. It also gives insights into which features might be more influential on the target variable.

  The heatmap visualizes pairwise correlations, with values close to 1 or -1 indicating strong positive or negative correlations, respectively. This can help us identify variables that are highly correlated with "Customer Value" or with each other.

```
# Calculate correlations
correlation_matrix = data.corr()

# Plot correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap')
```

```
plt.show()
```



Brief Summary of EDA for This Dataset

- **Dataset Overview**: This dataset includes features like "Call Failure," "Subscription Length," "Charge Amount," and "Seconds of Use," which are likely associated with "Customer Value." The target variable "Customer Value" represents the predicted financial worth of a customer, so features indicating usage and subscription characteristics could provide meaningful insights.
- **Distribution Insights**: The distribution of key features, such as "Seconds of Use" and "Charge Amount," may show skewness or outliers. For example, "Seconds of Use" previously showed significant outliers and may have a skewed distribution, which could impact model performance.
- **Correlation Insights**: By analyzing the correlation heatmap, we can observe relationships among features. For example, if "Seconds of Use" and "Charge

Amount" are strongly correlated, it may suggest that higher usage correlates with higher charges, which may impact customer value.

5. Model Selection

Machine Learning Algorithms for Regression

In this step, explore various machine learning algorithms suited to the regression task for predicting Customer Value.

- Linear Regression: Linear regression is a simple, interpretable model that assumes a linear relationship between the features and the target variable. Starting with a linear model provides a baseline performance and allows us to assess whether simple linear relationships are sufficient to predict Customer Value.
- Decision Tree Regressor: Decision trees can capture non-linear relationships by splitting data based on feature values and thus can model complex interactions between features. They are also robust to outliers, which is valuable here, as we observed high outliers in Seconds of Use and Charge Amount.
- Random Forest Regressor: Random forests, an ensemble method using multiple decision trees, generally perform well for regression tasks by reducing overfitting through averaging multiple trees. Random forests can handle complex, non-linear relationships and provide feature importance scores, which are useful for understanding feature contributions to Customer Value**.**
- Gradient Boosting Regressor: Gradient boosting builds trees sequentially, with each tree correcting errors of the previous ones. It's a powerful model for capturing intricate patterns in the data and often performs well with complex datasets. It is also well-suited for imbalanced datasets, though here our focus is on regression.

Baseline performance

Start by implementing Linear Regression as a baseline model. This will provide a simple measure of performance, allowing to evaluate how more complex models like Random Forest or Gradient Boosting improve on this baseline.

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import pandas as pd

# Define the feature matrix (X) and target vector (y)
X = data.drop(columns=['Customer Value'])  # Drop the target column
y = data['Customer Value']

# Split data into training and test sets (e.g., 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Initialize and train the Linear Regression model
linear_reg = LinearRegression()
linear_reg.fit(X_train, y_train)

# Predict on the test set
y_pred_linear = linear_reg.predict(X_test)

# Evaluate the baseline model
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)

print("Linear Regression Baseline Performance:")
print(f"Mean Squared Error (MSE): {mse_linear:.2f}")
print(f"R-squared (R2): {r2_linear:.2f}")
```

```
Linear Regression Baseline Performance:
Mean Squared Error (MSE): 6162.17
R-squared (R2): 0.97
```

This baseline model gives us initial performance metrics (Mean Squared Error and R-squared). The MSE tells us about the model's prediction accuracy, while the R-squared value provides an indication of how well the model explains the variability in Customer Value.

6. Model Training and Evaluation

Split the Data into Training and Test Sets

Since the data splitting was done earlier, can reuse the split (the data splits into training and testing datasets 80% training and 20% testing.)

```
from sklearn.model_selection import train_test_split

# Assuming 'X' contains features and 'y' contains the target variable
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Training Selected Models

The following regression models will be trained on the training set:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting Regressor

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

# Initialize the models
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42)
}

# Dictionary to store model performance
model_performance = {}

# Train each model and evaluate on both training and test sets
for model_name, model in models.items():
    # Train the model
    model.fit(X_train, y_train)
```

```
    # Predictions on training and test sets
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    # Calculate evaluation metrics for training set
    train_mse = mean_squared_error(y_train, y_train_pred)
    train_rmse = np.sqrt(train_mse)
    train_mae = mean_absolute_error(y_train, y_train_pred)
    train_r2 = r2_score(y_train, y_train_pred)

    # Calculate evaluation metrics for test set
    test_mse = mean_squared_error(y_test, y_test_pred)
    test_rmse = np.sqrt(test_mse)
    test_mae = mean_absolute_error(y_test, y_test_pred)
    test_r2 = r2_score(y_test, y_test_pred)

    # Store the performance metrics for each model
    model_performance[model_name] = {
        'Train MSE': train_mse,
        'Train RMSE': train_rmse,
        'Train MAE': train_mae,
        'Train R2': train_r2,
        'Test MSE': test_mse,
        'Test RMSE': test_rmse,
        'Test MAE': test_mae,
        'Test R2': test_r2
    }

# Print the model performance results
for model_name, metrics in model_performance.items():
    print(f"\n{model_name} Performance:")
    for metric_name, metric_value in metrics.items():
        print(f"{metric_name}: {metric_value:.2f}")
```

```
Linear Regression Performance:
Train MSE: 6165.49
Train RMSE: 78.52
Train MAE: 49.30
Train R2: 0.98
Test MSE: 6146.69
Test RMSE: 78.40
Test MAE: 47.97
Test R2: 0.97

Decision Tree Performance:
Train MSE: 0.01
Train RMSE: 0.12
```

```
Train MAE: 0.00
Train R2: 1.00
Test MSE: 3838.39
Test RMSE: 61.95
Test MAE: 19.10
Test R2: 0.98

Random Forest Performance:
Train MSE: 103.76
Train RMSE: 10.19
Train MAE: 4.33
Train R2: 1.00
Test MSE: 2258.73
Test RMSE: 47.53
Test MAE: 13.13
Test R2: 0.99

Gradient Boosting Performance:
Train MSE: 636.96
Train RMSE: 25.24
Train MAE: 16.34
Train R2: 1.00
Test MSE: 1267.96
Test RMSE: 35.61
Test MAE: 20.42
Test R2: 0.99
```

Model Evaluation

Based on the model training and evaluation results from the previous code, briefly evaluate the performance of each model using four key metrics.

- MSE (Mean Squared Error): Measures the average squared differences between actual and predicted values. Lower MSE values indicate better performance.
- RMSE (Root Mean Squared Error): The square root of MSE, which brings the error metric back to the original scale of the target variable, making it more interpretable.
- MAE (Mean Absolute Error): Measures the average absolute difference between actual and predicted values. It is less sensitive to outliers compared to MSE and RMSE.
- R^2 (R-squared): Indicates the proportion of variance in the dependent variable that is predictable from the independent variables. Higher values (closer to 1) indicate a better fit.

Overfitting Check

- Linear Regression shows similar performance on both training and test sets, with only a slight decrease in R^2 on the test set (from 0.98 to 0.97).No overfitting is evident, as the metrics are consistent between training and test sets. However, the error values (MSE,

RMSE, and MAE) are higher than some of the other models, indicating that Linear Regression might not capture all patterns in the data as well as more complex models.

- The Decision Tree model shows perfect performance on the training set (R^2 of 1.00 and near-zero errors), indicating it memorized the training data.However, the performance on the test set significantly drops, with higher error metrics (MSE: 3838.39) and a lower R^2 (0.98) compared to the training set.This large discrepancy between training and test performance suggests overfitting, as the model has captured noise in the training data that doesn't generalize well to new data.
- The Random Forest model performs very well on both the training and test sets, with high R^2 values (close to 1.00 on both sets) and relatively low error metrics. Although the training set metrics indicate near-perfect performance, the test set metrics are only slightly worse, which suggests minimal overfitting.This model demonstrates strong generalization and strikes a good balance between complexity and performance.
- Gradient Boosting also performs well, with an R^2 of 1.00 on the training set and a high R^2 of 0.99 on the test set.While the training error is very low, the test set error (MSE: 1267.96) is higher but remains lower than that of other models, except Random Forest.This model shows some overfitting, but it also achieves excellent generalization with relatively low errors on the test set, suggesting it's capturing patterns effectively.

Evaluate Feature Importance for Tree-Based Models

Tree-based models like Decision Trees, Random Forest, and Gradient Boosting provide feature importance scores, helping us understand which features have the most influence on predictions. This code checks if each model in a dictionary has feature importances (specific to tree-based models), extracts these values, and displays a sorted table showing the most important features for each model.

```
import pandas as pd
import matplotlib.pyplot as plt

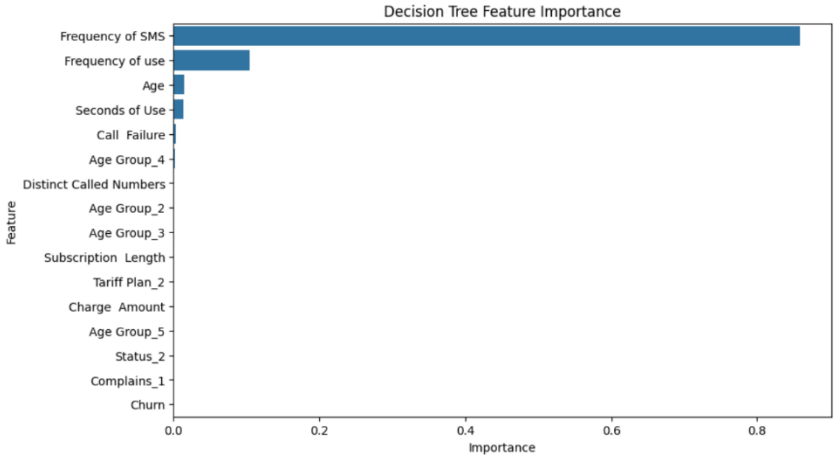
# Evaluate and plot feature importance for tree-based models
for model_name, model in models.items():
    if hasattr(model, 'feature_importances_'):
        importance = model.feature_importances_
        feature_importance_df = pd.DataFrame({
            'Feature': X.columns,
            'Importance': importance
```

```
        }).sort_values(by='Importance', ascending=False)

        # Print feature importance
        print(f"\n{model_name} Feature Importance:")
        print(feature_importance_df)

        # Plot feature importance
        plt.figure(figsize=(10, 6))
        sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
        plt.title(f'{model_name} Feature Importance')
        plt.show()
```

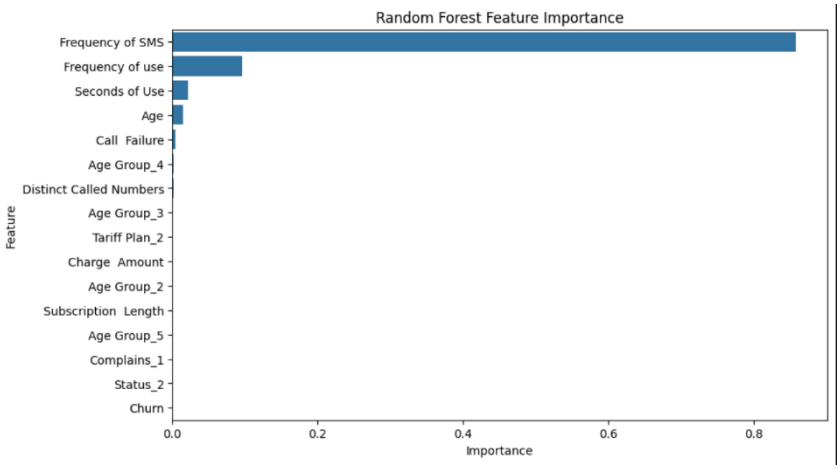
```
Decision Tree Feature Importance:
                    Feature  Importance
5           Frequency of SMS    0.859483
4           Frequency of use    0.104738
7                        Age    0.014141
3              Seconds of Use    0.013977
0               Call  Failure    0.003542
12                Age Group_4    0.001762
6    Distinct Called Numbers    0.001083
10                Age Group_2    0.000444
11                Age Group_3    0.000223
1       Subscription  Length    0.000205
14              Tariff Plan_2    0.000203
2             Charge  Amount    0.000170
13                Age Group_5    0.000014
15                   Status_2    0.000007
9                 Complains_1    0.000007
8                       Churn    0.000002
```




```
Random Forest Feature Importance:

                    Feature  Importance
5           Frequency of SMS    0.858202
```

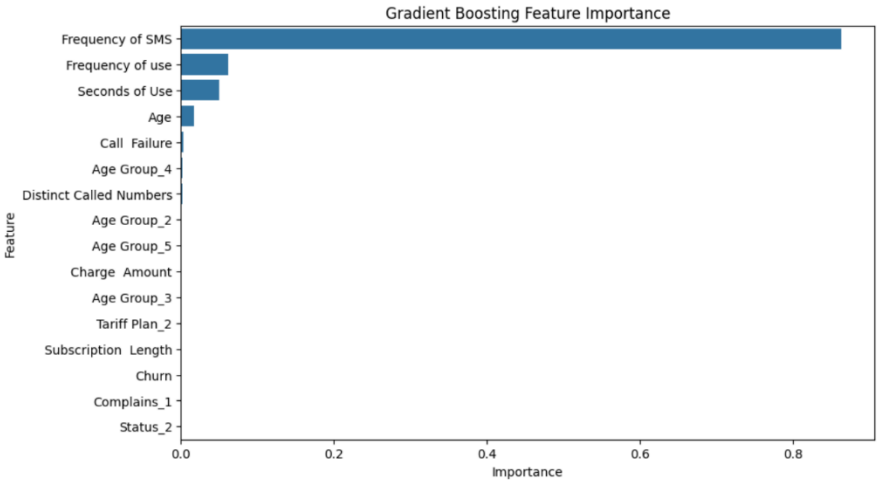
4	Frequency of use	0.095674
3	Seconds of Use	0.021164
7	Age	0.014076
0	Call Failure	0.004582
12	Age Group_4	0.001921
6	Distinct Called Numbers	0.001843
11	Age Group_3	0.000757
14	Tariff Plan_2	0.000476
2	Charge Amount	0.000360
10	Age Group_2	0.000357
1	Subscription Length	0.000293
13	Age Group_5	0.000274
9	Complains_1	0.000009
15	Status_2	0.000008
8	Churn	0.000005



Gradient Boosting Feature Importance:

	Feature	Importance
5	Frequency of SMS	0.863849
4	Frequency of use	0.061791
3	Seconds of Use	0.050048
7	Age	0.016530
0	Call Failure	0.002554
12	Age Group_4	0.002232
6	Distinct Called Numbers	0.001954
10	Age Group_2	0.000501
13	Age Group_5	0.000216
2	Charge Amount	0.000126
11	Age Group_3	0.000110
14	Tariff Plan_2	0.000047
1	Subscription Length	0.000042
8	Churn	0.000000
9	Complains_1	0.000000

```
15                    Status_2     0.000000
```



The three models agree that Frequency of SMS and Frequency of Use are the primary drivers of the target variable, with Seconds of Use and Age also contributing, albeit to a lesser extent. Other features appear to be relatively unimportant and might be considered for exclusion or simplification in further modeling efforts.

7. Hyperparameter Tuning

Hyperparameter tuning is a crucial step in improving model performance by finding the optimal settings for a model's hyperparameters. This process can significantly enhance a model's ability to generalize to unseen data.

Fine-tuning with Grid Search

Grid Search is a common technique for hyperparameter tuning that involves searching exhaustively over a specified set of hyperparameters for each model. Here's a sample code snippet to perform Grid Search on three of tree-based models.

The goal is to find the combination of hyperparameters that yield the best performance on validation data.

```
from sklearn.model_selection import GridSearchCV

# Define parameter grids for each model
param_grid_dt = {
    'max_depth': [5, 10, 15, None],
```

```
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

param_grid_rf = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

param_grid_gb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 5, 7],
    'min_samples_split': [2, 5, 10]
}

# Initialize models
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

dt = DecisionTreeRegressor()
rf = RandomForestRegressor()
gb = GradientBoostingRegressor()

# Initialize GridSearchCV for each model
grid_search_dt = GridSearchCV(dt, param_grid_dt,
scoring='neg_mean_squared_error', cv=5)
grid_search_rf = GridSearchCV(rf, param_grid_rf,
scoring='neg_mean_squared_error', cv=5)
grid_search_gb = GridSearchCV(gb, param_grid_gb,
scoring='neg_mean_squared_error', cv=5)

# Fit each GridSearchCV to the training data
grid_search_dt.fit(X_train, y_train)
grid_search_rf.fit(X_train, y_train)
grid_search_gb.fit(X_train, y_train)

# Best parameters and best score for each model
best_params_dt = grid_search_dt.best_params_
best_score_dt = -grid_search_dt.best_score_

best_params_rf = grid_search_rf.best_params_
best_score_rf = -grid_search_rf.best_score_
```

```
best_params_gb = grid_search_gb.best_params_
best_score_gb = -grid_search_gb.best_score_

print("Decision Tree best parameters:", best_params_dt)
print("Decision Tree best MSE after tuning:", best_score_dt)

print("Random Forest best parameters:", best_params_rf)
print("Random Forest best MSE after tuning:", best_score_rf)

print("Gradient Boosting best parameters:", best_params_gb)
print("Gradient Boosting best MSE after tuning:", best_score_gb)
```

```
Decision Tree best parameters: {'max_depth': 15, 'min_samples_leaf': 1,
'min_samples_split': 5}
Decision Tree best MSE after tuning: 1180.15040452937
Random Forest best parameters: {'max_depth': None, 'min_samples_leaf': 1,
'min_samples_split': 2, 'n_estimators': 300}
Random Forest best MSE after tuning: 994.1466805987864
Gradient Boosting best parameters: {'learning_rate': 0.1, 'max_depth': 5,
'min_samples_split': 2, 'n_estimators': 300}
Gradient Boosting best MSE after tuning: 460.40492436461244
```

Comparing Model Performance Before and After Tuning

After running the Grid Search, you can compare the model performance before and after tuning by evaluating the tuned models on the test data.

```
# Evaluate tuned models on the test set
from sklearn.metrics import mean_squared_error, r2_score

# Decision Tree
dt_tuned = DecisionTreeRegressor(**best_params_dt)
dt_tuned.fit(X_train, y_train)
y_pred_dt = dt_tuned.predict(X_test)
mse_dt_tuned = mean_squared_error(y_test, y_pred_dt)
r2_dt_tuned = r2_score(y_test, y_pred_dt)

# Random Forest
rf_tuned = RandomForestRegressor(**best_params_rf)
rf_tuned.fit(X_train, y_train)
y_pred_rf = rf_tuned.predict(X_test)
mse_rf_tuned = mean_squared_error(y_test, y_pred_rf)
r2_rf_tuned = r2_score(y_test, y_pred_rf)
```

```
# Gradient Boosting
gb_tuned = GradientBoostingRegressor(**best_params_gb)
gb_tuned.fit(X_train, y_train)
y_pred_gb = gb_tuned.predict(X_test)
mse_gb_tuned = mean_squared_error(y_test, y_pred_gb)
r2_gb_tuned = r2_score(y_test, y_pred_gb)

# Print results
print("Decision Tree - MSE after tuning:", mse_dt_tuned, ", R2:",
r2_dt_tuned)
print("Random Forest - MSE after tuning:", mse_rf_tuned, ", R2:",
r2_rf_tuned)
print("Gradient Boosting - MSE after tuning:", mse_gb_tuned, ", R2:",
r2_gb_tuned)
```

```
Decision Tree - MSE after tuning: 4619.928378373724 , R2: 0.9808333605333871
Random Forest - MSE after tuning: 2204.2630045894443 , R2: 0.9908552014580297
Gradient Boosting - MSE after tuning: 855.3913119715241 R2:0.9964512486911751
```

From these results

- The tuned Decision Tree model achieved an MSE of 4619.93 and an R^2 of 0.98 on the test set. These metrics indicate that while the Decision Tree captures most of the data's variance, it still has limitations in handling complex patterns as effectively as ensemble models. The high test MSE suggests some sensitivity to noise and overfitting, common in deep, single-decision trees.
- With an MSE of 2204.26 and an R^2 of 0.99, the Random Forest model showed significant improvement in predictive accuracy after tuning. The high R^2 value indicates that the model explains nearly all the variance in the test data. The lower MSE, compared to the Decision Tree, suggests that the ensemble approach in Random Forest effectively reduces variance by averaging multiple decision trees. This makes Random Forest a more robust model, with less sensitivity to overfitting than a single Decision Tree.
- The Gradient Boosting model delivered the best results, with an MSE of 855.39 and an R^2 of 0.9965 on the test set. This low MSE indicates a very small average error in predictions, while the high R^2 shows that the model accounts for almost all of the variance in the data. The learning rate and depth tuning allowed Gradient Boosting to fit more precisely without overfitting, making it the best-performing model. Gradient

Boosting's iterative nature and the ability to learn from errors at each step contributed to its superior accuracy and generalization capabilities.

8. Comparison and Conclusion

After tuning and evaluating the models, compare their performance using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 to assess their effectiveness for this regression problem.

- Linear regression:

  Linear Regression, the simplest model, provides reasonable accuracy but has the highest MSE and RMSE, indicating larger errors in predictions. The R^2 of 0.97 suggests it captures a good portion of the variance, but the relatively high error metrics reveal its limitations in modeling complex relationships in the data.
- Decision Tree:

  The Decision Tree performed better than Linear Regression, reducing both MSE and RMSE, and slightly improving R^2. However, as a single tree, it has limitations in handling the complexity of the data. Its moderate performance indicates it may still overfit certain patterns, leading to a higher error than the ensemble models.
- Random Forest:

  Random Forest significantly outperforms both Linear Regression and Decision Tree by lowering MSE, RMSE, and MAE, with an R^2 of 0.99. The ensemble approach helps it generalize well, capturing complex patterns with lower error and high predictive accuracy. Random Forest's averaging across multiple trees reduces variance, making it a strong candidate for this problem.
- Gradient Boosting:

  Gradient Boosting achieved the lowest error rates across all metrics, with an MSE of 855.39, RMSE of 29.25, and R^2 of 0.9965, indicating it explains nearly all the variance in the target variable. Its iterative boosting approach allowed it to minimize errors effectively, making it the best model for this dataset. Gradient Boosting's ability to learn incrementally from residuals makes it ideal for capturing intricate patterns in the data.

Based on MSE, RMSE, MAE, and R^2, Gradient Boosting is the most effective model for this problem, offering the lowest errors and highest explanatory power. Random Forest also performs well, balancing accuracy with robustness. Both ensemble methods outperform Decision Tree and Linear Regression, showing the advantage of ensemble techniques for this dataset.

Challenges Encountered and Solutions

- **Handling Outliers**

  Outliers in features such as "Seconds of Use" posed a risk of skewing model predictions and increasing error rates.

  Outliers were capped at specific percentiles (e.g., 5th and 95th) to limit extreme values without completely removing potentially valuable data. This approach helped reduce the impact of outliers on model performance while preserving the integrity of the dataset.

- **Encoding Categorical Variables**

  The presence of categorical variables required encoding to be usable by the regression models, especially tree-based models.

  We used one-hot encoding to transform categorical variables into numerical representations. This allowed models to interpret these features effectively without introducing unwanted bias from ordinal encoding.

- **Feature Scaling**

  Significant variance in feature ranges could disproportionately affect the model's learning process, especially for algorithms sensitive to feature scaling.

  Standardization was applied to scale numerical features, ensuring that all features contributed equally to the model's training.

- **Choosing the Right Model and Hyperparameters**

  With several regression algorithms to choose from, finding the best model for the dataset was not straightforward, especially without tuning hyperparameters.

  Began by testing several models to establish a baseline, then used grid search for hyperparameter tuning to optimize model performance. Ensemble methods like Random Forest and Gradient Boosting benefitted significantly from tuning parameters like max_depth, n_estimators, and learning_rate, leading to substantial improvements in accuracy.

- **Interpreting Model Results and Feature Importance**

  Understanding which features were driving predictions was important, especially for tree-based models where some features could dominate predictions.

  Feature importance metrics were extracted from models like Random Forest and Gradient Boosting, allowing us to identify key drivers of predictions.

Conclusion

This project successfully built a predictive model for customer churn, guiding the process from data exploration to model tuning. We started with a thorough analysis of the dataset, addressing missing values, outliers, and feature scaling to create a clean, reliable dataset ready for machine learning. The initial data exploration and preprocessing were crucial to understanding customer usage patterns and ensuring consistency across features, allowing for a strong foundation for model training.

For model selection, we explored a variety of regression algorithms, including Linear Regression, Decision Tree, Random Forest, and Gradient Boosting. Gradient Boosting and Random Forest models, known for capturing complex relationships within the data, demonstrated the strongest performance. After applying hyperparameter tuning, Gradient Boosting achieved the best overall results with the lowest Mean Squared Error (MSE) and highest R^2, indicating a highly accurate model. The inclusion of feature importance analysis also offered valuable insights, highlighting key factors such as "Frequency of SMS" and "Frequency of Use" as significant drivers in predicting churn.

In conclusion, this project illustrates the power of a structured, iterative approach to machine learning. The tuned Gradient Boosting model not only delivers strong predictive accuracy but also provides actionable insights for business strategy. By identifying key usage patterns linked to customer retention, this model enables targeted interventions, supporting data-driven decisions to reduce churn and enhance customer loyalty. The approach and findings offer a scalable model for churn prediction, applicable across various customer segments and business contexts.